Model width and delayed modular generalization: a finite-horizon study with recipe boundaries

Anonymous Author(s)
Affiliation
Address
email

Abstract

How does model size change delayed generalization on small algorithmic tasks? We test the explanation that a wider trainable feature basis advances shared-structure learning while preserving a long interval after training fit. Across 204 registered CPU runs of one-hidden-layer ReLU networks, the main addition-modulo-31 recipe has held-out onset counts of 0, 0, 8, 12 and 12 out of twelve at widths 32 through 512. The paired endpoint restricted-onset difference is -43800 updates (95% interval $[-44150, -43433]$), and all twelve wide runs retain the declared substantial delay. A frozen-feature discriminator is unresolved because its narrow models cannot fit. Endpoint width advantages persist under both alternative learning rates and multiplication, but lower decay produces no primary onset at any tested width despite a large horizon-accuracy difference. Orbit-split results depend on the held-out threshold, and confirmed onset need not imply terminal success. These findings establish a recipe- and window-dependent size relationship; they do not establish the complete feature-learning explanation, an eventual learning-time law or a unique mechanism.

1 Introduction

On small algorithmically generated tasks, fitting the training labels can precede held-out prediction by a long interval. This delayed generalization, or grokking, makes training accuracy an incomplete description of learning [Power et al., 2022]. Prior experiments already vary transformer size and training duration on modular division and report model-wise grokking [Davies et al., 2023]. We ask which size-dependent behavior and explanatory consequences hold in a precisely specified MLP system.

Our candidate explanation is that a wider trainable feature basis advances shared-structure learning while preserving a substantial interval after fit. Its consequences must be separated: earlier held-out onset, persistent post-fit delay, and a width benefit attributable to hidden trainability. A shorter generalization clock does not establish a shorter delay without the fit clock; a frozen model that cannot fit does not isolate feature learning from fitting capacity.

We test three prospective predictions on addition modulo 31, pairing fresh split/initialization blocks across five widths. All 204 registered runs include feature freezing, decay and learning-rate changes, unordered-orbit splits and separately trained multiplication models. We retain censored clocks, onset/terminal differences and seed-level uncertainty. The main comparison supports earlier restricted generalization and persistent delay, but the frozen fit gate fails (Section 5). Complete controls expose recipe and threshold boundaries (Section 6). The contribution is this controlled characterization and bounded explanatory test, rather than a first-size-study claim, a parameter-count law or a unique mechanism.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

2 Related work

Algorithmic tasks, size and time. Power et al. [2022] introduce small operation-table tasks with delayed held-out improvement and dependence on data fraction and training choices. Davies et al. [2023] directly study parameter count and training time in modular-division transformers, including model-wise grokking and nonmonotone behavior. Our MLP study separates by-horizon incidence, restricted clocks and post-fit delays with seed uncertainty. Prior size studies preclude an exhaustive-priority claim.

Norm, efficiency and feature learning. Liu et al. [2023] analyze norm-conditioned loss landscapes and constrained-norm training. Varma et al. [2023] propose competing circuits with different generalization, learning speed and logit efficiency per norm. A total-norm trajectory does not reproduce either identifying comparison. Kumar et al. [2024] give no-decay/increasing-norm examples and analyze a lazy-to-rich transition; larger width under common parameterizations can also improve the lazy approximation. Our width prediction is empirical, rather than a deduction that wider always means richer dynamics. A frozen readout also differs from full-network initial linearization, which retains derivatives for every trained parameter direction. Freezing removes hidden shrinkage and changes the trainable count and geometry, limiting adaptation-only interpretations.

Functional structure and regularization. Nanda et al. [2023] identify Fourier algorithms in modular-addition transformers through internal weights, activations and targeted ablations. Our Fourier logit-energy fraction is an output-functional diagnostic, not their circuit identification, causal mediator or a multiplication-circuit measurement. AdamW decouples weight decay from adaptive gradient updates [Loshchilov and Hutter, 2019]; changing its learning rate at a fixed decay coefficient also changes per-update shrinkage. Recipe controls therefore constrain a bounded explanation without establishing an optimizer-independent size law.

3 Explanation and falsifiable predictions

The candidate explanation concerns a fixed training recipe: increasing the width of a trainable feature basis can accelerate learning of shared arithmetic structure while leaving a substantial interval after training-set fit. It predicts an earlier generalization event at larger width, rather than merely earlier memorization. This is an explanation to test, not an established capacity law. Changing width also changes initialization and optimization dynamics; the experiment does not isolate parameter count from those changes.

The prospective protocol was fixed after exploratory regime-finding runs and before the confirmatory seed blocks. Its three predictions have distinct evidential requirements. Let T_m and T_g denote confirmed training-fit and held-out generalization onsets, and let $R_g = \min(T_g, H)$ at the common horizon H; Section 4.3 specifies their operational definitions.

P1: earlier restricted generalization. Under the main recipe, the paired mean contrast

$$\Delta_g = \mathbb{E}[R_g(512) - R_g(32)] \tag{1}$$

is negative. A confidence interval wholly below zero supports the directional prediction; an interval wholly above zero contradicts it. An interval spanning zero, or pervasive joint censoring, leaves it unresolved. The full five-width response is reported even if it is nonmonotone.

P2: delay persists at the larger width. More than half of width-512 runs exhibit confirmed generalization after fit with both $T_g - T_m \geq 1000$ updates and $T_g/T_m \geq 10$. These cutoffs define a substantial delay for this study, rather than a natural boundary of grokking. A 95% Wilson lower bound above 0.5 supports P2; an upper bound below 0.5 contradicts it; otherwise it remains unresolved. Continuous delays, censored lower bounds and threshold sensitivities accompany this classification.

P3: feature freezing attenuates the width advantage. For the same first eight seed blocks, define $D_c = \mathbb{E}[R_g(512, c) - R_g(32, c)]$, where c is full training or frozen hidden features. The prediction is $D_{\text{full}} - D_{\text{frozen}} < 0$. The representation-versus-fitting interpretation additionally requires at least

six of eight frozen runs at *each* endpoint width to reach training fit; the corresponding full-model counts must also be reported. Failure of this gate leaves that interpretation unresolved, even if the restricted-clock interaction has the predicted sign. Freezing changes both hidden adaptation and hidden weight shrinkage, so this intervention cannot separate their contributions.

Alternative outcomes matter: width may accelerate only fitting, the response may depend on the optimizer recipe, or frozen features may exhibit a comparable width advantage. Support for the complete account requires P1, P2 and an interpretable P3. Mixed outcomes warrant a conditional or revised explanation while retaining the original predictions. Neither a width sweep alone nor a functional Fourier diagnostic identifies a unique internal circuit.

4 Registered method

The protocol follows exploratory regime finding and precedes the fresh confirmatory blocks. Pilots are excluded from confirmatory estimates. All 204 registered runs completed; the fixed design and its execution record are reported separately.

4.1 Tasks, models and training

Enumerate all 31^2 ordered pairs and predict $(a + b) \bmod 31$. A seeded permutation assigns the first $\lfloor 0.5 \times 31^2 \rfloor$ pairs to training and the complement to held-out evaluation. Reverse pairs can cross this split. The orbit control instead shuffles unordered orbits $\{(a, b), (b, a)\}$ and assigns half, rounded down, wholly to training; its realized pair fraction varies. Multiplication retains the same domain with label $(ab) \bmod 31$, recording zero-related and nonzero held-out subsets separately. Data are generated exactly in memory, without an external download.

A biased one-hidden-layer ReLU classifier maps concatenated one-hot inputs to 31 logits. Widths $32, 64, 128, 256, 512$ have total parameter count $N(w) = (3p+1)w+p$, with $p = 31$ (Table 1). Summing the two indexed input-weight columns exactly implements the one-hot affine map; operand weights are not tied. Initialization follows the PyTorch Linear uniform convention (Appendix A.1). Frozen-hidden controls retain their initialized hidden weights/biases and train only the readout. Shared seed labels do not match initialized functions across widths of different shapes.

Training uses full-batch cross-entropy and AdamW: learning rate 10^{-3}, decay 1, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, no scheduler, and decay on all trainable parameters including biases. Frozen hidden parameters receive neither gradient updates nor shrinkage. CPU execution uses deterministic PyTorch algorithms, one intra-operation thread per run and at most two concurrent runs. Exact versions and recovery/accounting details are in Appendix A.4 and the reproduction package.

4.2 Replication and boundaries

Main block $j = 0, \ldots, 11$ uses split seed $1100 + j$ and initialization seed $2100 + j$, paired across all five widths. Each secondary family uses the first eight blocks at widths $32, 128, 512$: decay 0.1, frozen hidden features, learning rate 3×10^{-4} or 3×10^{-3}, orbit splitting, or multiplication. The complete design specifies 204 runs. Within a task/split rule, split hashes must match across paired conditions. A block varies split and initialization together; uncertainty does not separate their contributions. Learning-rate changes also change per-update shrinkage; orbit splitting changes the realized pair fraction as well as reverse-pair access.

4.3 Events and finite-horizon outcomes

Evaluate at initialization and every 100 updates. Training-fit onset T_m is the first evaluation at training accuracy at least 0.99 followed by two scheduled evaluations meeting that threshold; held-out onset T_g uses 0.95 with the same persistence. Onset and confirmation are distinct. The onset horizon is $H = 60000$; execution reaches 60200 so an onset at H can be confirmed. No performance-based early stopping is used. Terminal success separately checks the final three evaluations; neither rule asserts threshold maintenance between samples.

An unobserved onset remains censored, with a flag and null event time. The restricted clocks R_m, R_g use H for noncrossings without imputing a crossing. Restricted waiting is $\max(0, R_g -$

Table 1: All main widths: instantiated total/trainable parameters, training-fit, generalization and delayed-event counts, and mean restricted generalization time R_g in *thousand updates* with 95% seed-bootstrap intervals. All twelve blocks remain in every denominator; horizon values are restrictions, not observed onsets.

Width	Params	Trainable	Fit	Gen.	Delay	R_g, mean [95% CI]
32	3,039	3,039	12/12	0/12	0/12	60.0 [60.0, 60.0]
64	6,047	6,047	12/12	0/12	0/12	60.0 [60.0, 60.0]
128	12,063	12,063	12/12	8/12	8/12	42.0 [35.1, 49.3]
256	24,095	24,095	12/12	12/12	12/12	18.6 [18.1, 19.1]
512	48,159	48,159	12/12	12/12	12/12	16.2 [15.8, 16.6]

R_m); conditional observed-event delays and fitted-noncrosser lower bounds retain their denominators (Appendix A.2). Completed noncrossings, unusual ordering, undefined zero-clock ratios, process failures and unstarted registrations remain distinct statuses.

4.4 Uncertainty and diagnostics

P1 uses all twelve paired blocks; P3 resamples eight complete four-cell blocks. Report 95% paired percentile-bootstrap intervals from 10000 draws, seed 1701; proportions use Wilson 95% intervals. Exhaustive held-out evaluation does not create independent training replications. Small-block bootstrap intervals are approximate, and joint censoring can produce degenerate restricted contrasts without proving eventual equality. Secondary paired binary intervals retain discordance tables and conservative exact bounds (Appendix A.2); secondary intervals are pointwise.

Sensitivities cover train thresholds $\{0.95, 0.99\}$, held-out thresholds $\{0.90, 0.95, 0.99\}$, persistence $\{1, 3\}$, and the primary definition thinned to 200-update observations. They retain the primary decision rule. Diagnostics record loss, norm/displacement and addition-compatible Fourier logit energy, a descriptive output-functional measure (Appendix A.3). Efficiency uses the last measured evaluation within H at or before 100 process-CPU seconds, with actual sampled steps and timing exclusions; it is distinct from the fixed-update estimand. Attempts, historical pilots, hashes and timing records are retained. No optional post-horizon follow-up was performed.

5 Completed primary and random-feature experiments

All 204 registered runs completed through 60200 updates, and the independent raw-record audit passed for all runs. This section reports the 60 main and 24 random-feature runs; Section 6 reports all 120 recipe/split/operation controls. Figure 1 and Table 1 retain every main width and denominator; supporting trajectories and every seed clock appear in Appendix B.

5.1 Width changes held-out learning within the window

All twelve main runs at each width reach training fit. Confirmed generalization counts at widths $32, 64, 128, 256, 512$ are respectively $0, 0, 8, 12, 12$ out of twelve; delayed-event counts are the same. Thus fitting is insufficient for generalization by the common horizon, and the observed occurrence of delayed generalization increases across this width grid. Width 128 retains four noncrossings rather than an imputed eventual onset. Onset and terminal sustained-generalization counts agree in these completed cells, without implying continuous success between evaluations or beyond execution.

For P1, the paired width 512-minus-32 restricted generalization-time difference is -43800 updates (95% paired-bootstrap interval $[-44150, -43433]$, twelve blocks). This supports its prespecified negative direction. Width 512's mean observed onset is 16200 updates ($[15850, 16567]$); width 32 contributes restriction at 60000 in every block. This is a finite-window contrast; width 32's degenerate restricted interval does not locate its eventual onset.

Training fit is also earlier at larger width: the endpoint paired restricted-fit difference is -1742 updates ($[-1817, -1675]$). The paired restricted waiting-time difference remains -42058 updates ($[-42408, -41717]$), so the observed generalization-clock contrast cannot be described solely as

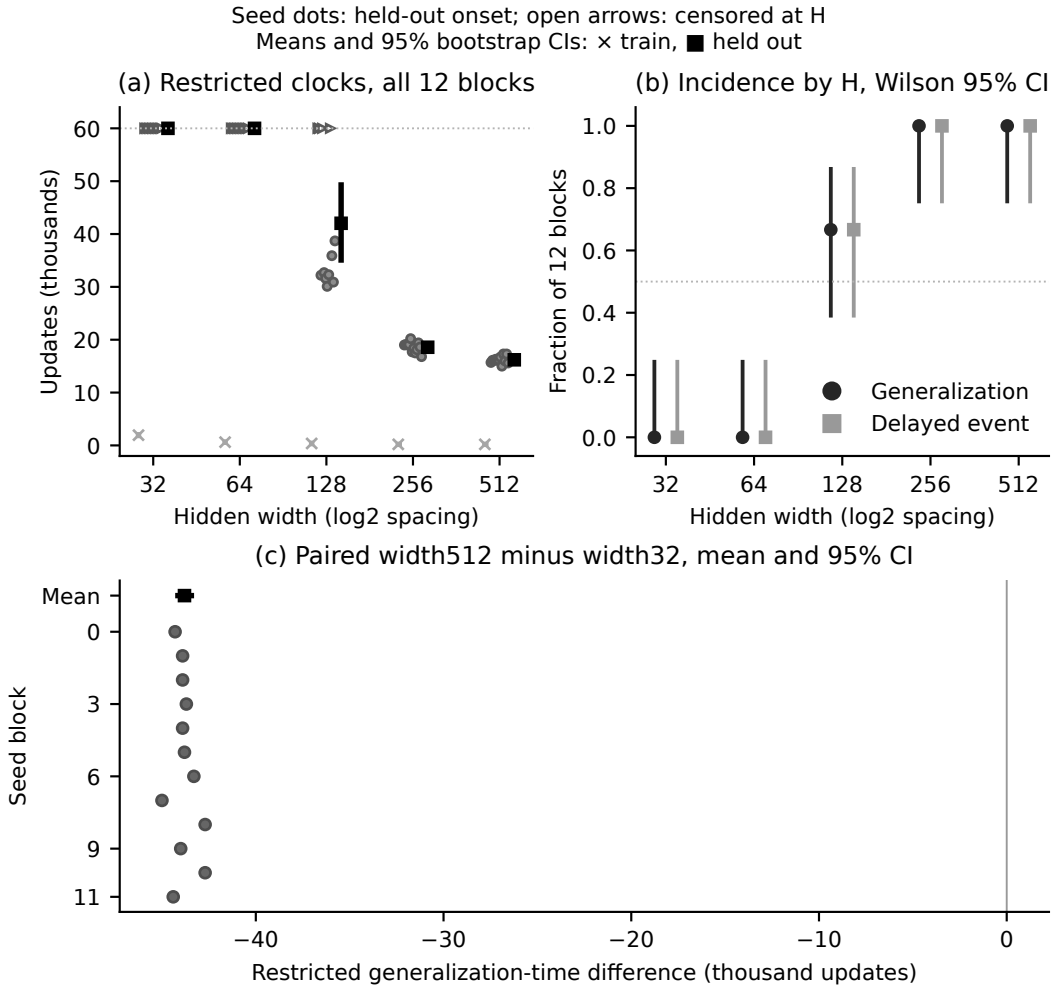


Figure 1: Main-recipe width comparison, all twelve blocks. (a) Seed held-out onsets, restricted train/held-out means and 95% seed-bootstrap intervals. Open arrows mark non-events restricted at $H = 60000$, rather than observed crossings. (b) Generalization and delayed-event fractions with Wilson 95% intervals. (c) All paired width-512-minus-32 restricted generalization-time differences and their mean/95% paired interval. Collapsed intervals under common censoring do not establish eventual equality.

moving the fit clock earlier. This does not establish that eventual post-fit delay is shorter for all narrow models, whose generalization times remain censored.

5.2 Substantial delay persists at width 512

P2 is supported: all twelve width 512 blocks meet the registered lag and ratio criteria, giving delayed-event prevalence $12/12$ with Wilson 95% interval $[0.758, 1]$. The lower bound exceeds the declared majority boundary. Every run fits at update 200 on the sampled grid, while the mean post-fit wait is 16000 updates ($[15650, 16367]$). These data show earlier generalization alongside a substantial remaining delay. Training-fit accuracy does not identify an internal memorization circuit.

5.3 The frozen control does not resolve the explanation

Figure 3 retains the matched intervention display in the appendix. In the first eight matched blocks, the full-minus-frozen endpoint interaction is -43975 updates ($[-44325, -43688]$). Its statistical direction agrees with P3, but its prospective fit gate fails. The frozen width 32 models fit in $0/8$ runs

and frozen width 512 in $8/8$, compared with $8/8$ at each full-model endpoint; frozen width 128 also fits in $0/8$. No frozen width has a confirmed generalization event in any of its eight runs.

Thus the narrow frozen model cannot supply the planned fit-comparable discriminator. At width 512 the frozen model can fit yet does not generalize by the horizon, providing a within-width observation consistent with the importance of hidden trainability under this recipe. It does not rescue the failed across-width gate or distinguish adaptation from hidden decay, optimization geometry and trainable count. The complete candidate explanation is therefore *not established*: P1 and P2 are supported, while the representation-versus-fitting interpretation of P3 remains unresolved.

5.4 Definition sensitivity and the compute estimand

All thirteen prescribed sensitivity variants preserve the negative P1 interval, P2's majority decision and P3's failed fit gate on these completed families. The held-out threshold changes the measured clock contrast; it does not select a new primary definition. The 200-update thinned view only removes existing evaluations and has reduced persistence-confirmation coverage near the endpoint. Scope-family sensitivities are reported below and in Appendix C.

The update-based ranking differs from a fixed measured CPU budget. At the last measured evaluation at or before 100 process-CPU seconds, width 128's mean held-out accuracy is 92.26% ($[91.18, 93.26]$) versus 61.17% ($[58.25, 64.02]$) at width 512, with twelve available blocks in each cell. The actual sampled steps span 27700–31200 and 9700–10100, respectively; no interpolation to an exact 100-second value is performed. These are descriptive all-block summaries on this implementation and host, rather than a paired test of an optimal width or a hardware-independent efficiency law.

6 Recipe, split and operation boundaries

Each control cell contains eight blocks; endpoint differences below are width 512 minus 32 with pointwise 95% paired-bootstrap intervals. Figure 2 retains all six conditions, including noncrossings; Appendix C gives every cell, contrast, sensitivity and diagnostic.

Learning rate and decay qualify the width relationship. Both alternative rates retain an earlier wide endpoint: the restricted-onset differences are -28112.5 updates ($[-29650, -26650]$) at 0.0003, and -48375 ($[-48712.5, -48050]$) at 0.003. Both endpoint intervals stay wholly negative in all thirteen definitions. At the low rate, widths 32/128 fit but have $0/8$ primary onsets, while width 512 has $8/8$; at the high rate, width 32 has $0/8$ and widths 128/512 have $8/8$. Onset and terminal counts agree at the primary threshold, but the medium high-rate cell has only $2/8$ onsets and $0/8$ terminal successes at 99%; wide low-rate counts fall to $5/8$ at 99%.

At decay 0.1, all three widths fit yet have $0/8$ primary onset or terminal successes (each Wilson interval $[0, 0.324]$). The endpoint restricted-time difference is $0[0, 0]$ from common restriction, although H-accuracy differs by $+85.21$ percentage points ($[83.81, 86.62]$). Thus the primary event-clock advantage does not extend to this recipe within the window; it is not a zero accuracy effect or equality of eventual times. Matched decay 0.1-minus-1 onset-incidence differences at widths 32/128/512 are 0, -0.625, -1; only the wide exact interval excludes zero ($[-1, -0.156]$). Restricted-time differences are 0, $+17637.5$ ($[7100, 25525]$) and $+43975$ ($[43687.5, 44325]$) updates. A wide single-point 90% onset fails terminal 90%; three-point 90% gives none. These comparisons do not estimate a width-by-recipe interaction or isolate gradient speed from AdamW shrinkage.

Split and operation boundaries retain failures. Under the unordered-orbit split, widths 32/128 have $0/8$ primary onsets; width 512 has $5/8$ (Wilson $[0.306, 0.863]$), but only $4/8$ terminal successes ($[0.215, 0.785]$). Its endpoint restricted contrast is -18812.5 updates ($[-28487.5, -7975]$). This does not establish majority delayed prevalence, and at 99% both endpoint event counts are zero, giving restricted difference $0[0, 0]$. Removing shared unordered orbits also changes the realized ordered-pair training fraction; this is not an isolated causal reverse-pair-sharing comparison.

For multiplication, onset counts at widths 32/128/512 are $0, 8, 8$ and terminal counts $0, 7, 8$ out of eight. The endpoint restricted difference is -43425 updates ($[-44350, -42450]$), with negative intervals throughout the thirteen definitions. The medium primary onset without terminal success

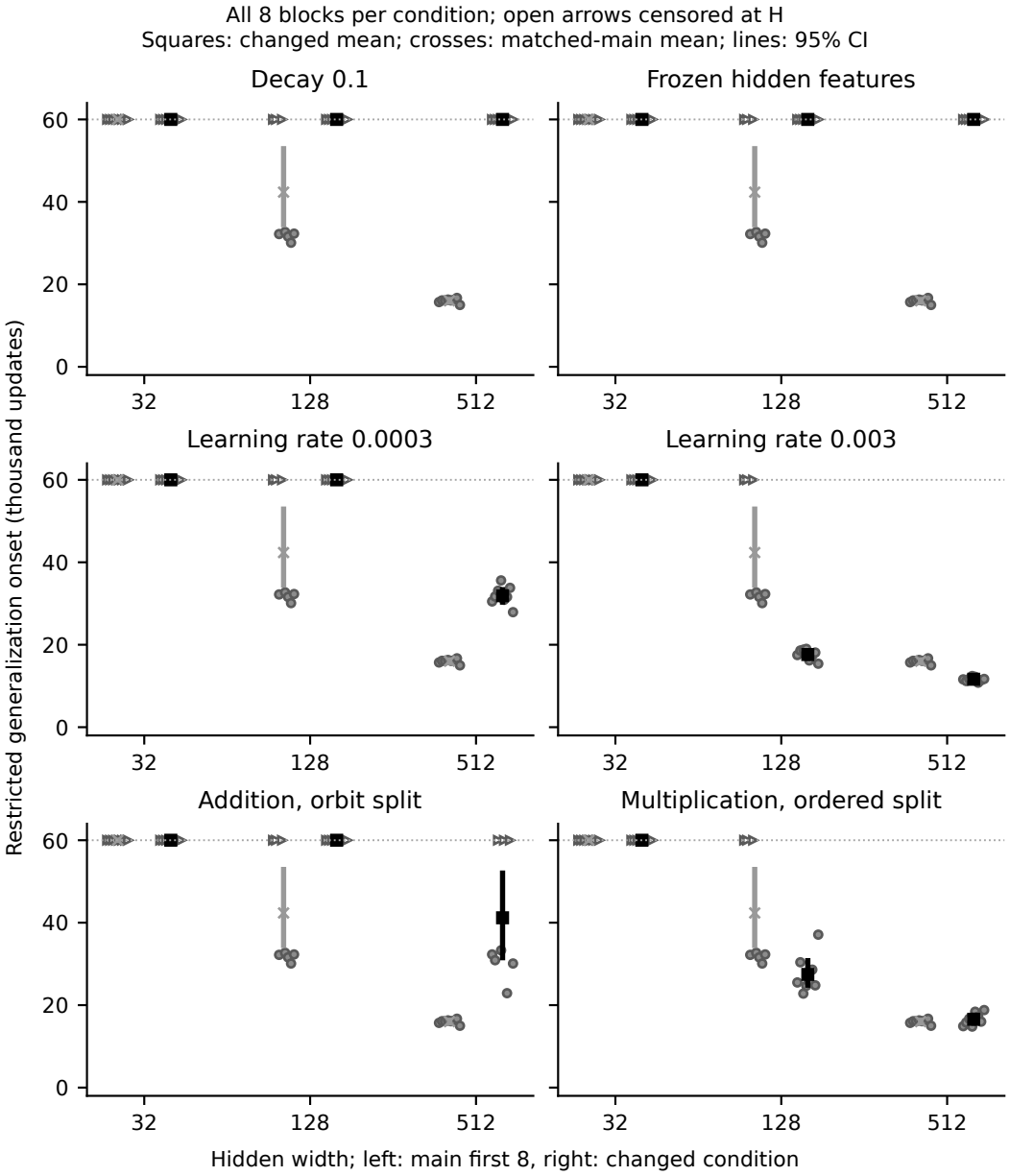


Figure 2: All six registered control conditions, eight blocks at each width. Individual held-out onset clocks retain non-events restricted at H; cell means and 95% seed-bootstrap intervals summarize this finite window. The matched main comparator uses its first eight blocks. Recipe/split/operation changes have different boundaries: common restriction under low decay and freezing does not establish equal eventual learning or absence of an accuracy effect. Onset and terminal success are separately reported in the text and appendix.

shows that confirmation does not imply permanent generalization. All runs reach perfect H-accuracy on zero-input cases, but wide nonzero accuracy is also high: 99.56% ($[99.42, 99.72]$), versus 44.76% ($[35.46, 54.16]$) at width 32. These are seed intervals, not independent input-pair replications or evidence of a shared addition circuit.

7 Discussion

The main answer is conditional: within this parameterization, recipe and horizon, larger width makes delayed generalization more frequent and moves its held-out clock earlier, while leaving a long post-fit interval. Incidence, restricted onset, waiting and terminal success answer different questions. The learning-rate and multiplication controls extend the observed endpoint advantage, while low decay saturates the primary event clock and orbit splitting makes the conclusion threshold-dependent. A size recommendation therefore needs an explicit recipe, criterion and observation window.

P1 and P2 support the descriptive consequences of the candidate explanation. The much larger restricted-wait contrast than fit-clock contrast rules out an account solely in terms of earlier training fit. P3 nevertheless fails its planned fit-comparability gate. Frozen wide models can fit without generalizing, but their narrow counterparts cannot fit; hidden adaptation, shrinkage and optimization geometry remain confounded. The complete feature-learning explanation is unresolved, and Fourier or norm trajectories cannot select a unique mechanism. Further identifying interventions would need fit-comparable controls and a parameterization that separates those changes.

Budget choice also changes the empirical question. At a common update horizon, width can improve held-out learning; at 100 measured process-CPU seconds, wider runs have made fewer updates, and primary width 128 outperforms 512 descriptively. Low decay similarly has a large positive H-accuracy endpoint contrast but a small negative CPU-sampled one. These are distinct estimands, with unequal progress, early completion and one unavailable timing record explicitly retained. They establish neither an optimal width nor a hardware-independent scaling law.

8 Limitations and research process

Width varies within one biased ReLU MLP family, at one prime and nominal data fraction. It changes capacity, initialized functions and optimization together; matched seed labels do not match functions across widths. The frozen readout is not the full initial tangent model and removes hidden decay. Two positive decay coefficients cannot establish decay's necessity. No prime/depth/activation/fraction sweep, causal circuit ablation, NTK measurement or post-horizon continuation was performed.

Twelve main and eight control blocks limit precision. Split and initialization vary jointly; intervals cannot separate their contributions. Bootstrap intervals are approximate and sometimes degenerate under common restriction; secondary comparisons are pointwise. Sampled thresholds/persistence define onset operationally, without continuous observation or eventual-time identification. Confirmed onset can fail terminal success. Orbit splitting changes realized pair fractions, and multiplication has an easy zero subrule. Pilots selected the recipe before fresh confirmatory blocks; the complementary held-out set is not an untouched external task.

All registered experiments completed, but one interrupted/resumed execution has incomplete CPU accounting. The recovery, failed guards, original missed interim deadline and revised schedule are retained (Appendix A.4); reported compute is a recorded lower bound. Research agents contributed to design, implementation, analysis, presentation and writing; the code and recorded checks support inspection but are not human validation (Appendix A.7). Permitted literature was actually consulted within recorded scopes; prohibited prior work was not sought, and no exhaustive novelty audit is claimed.

The synthetic-task findings may help choose reproducible experimental regimes, but do not establish deployed-model capabilities, privacy or fairness effects. Direct misuse risk is limited for these small modular classifiers; broad extrapolation of their size relationship would be a potential misleading use. No personal data, crowdsourcing or human subjects were involved.

References

Xander Davies, Lauro Langosco, and David Krueger. Unifying grokking and double descent. *arXiv preprint arXiv:2303.06173*, 2023. URL https://arxiv.org/pdf/2303.06173v1.

Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan. Grokking as the transition from lazy to rich training dynamics. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/pdf/2310.06110v3.

Ziming Liu, Eric J. Michaud, and Max Tegmark. Omnigrok: Grokking beyond algorithmic data. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/pdf/2210.01117v2.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/pdf/1711.05101v3.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/pdf/2301.05217v3.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022. URL https://arxiv.org/pdf/2201.02177v1.

Vikrant Varma, Rohin Shah, Zachary Kenton, János Kramár, and Ramana Kumar. Explaining grokking through circuit efficiency. *arXiv preprint arXiv:2309.02390*, 2023. URL https://arxiv.org/pdf/2309.02390v1.

A Operational quantities and reproduction details

This appendix documents all 204 completed registered runs and the analysis implementation. The canonical experiment commands, environment, attempt inventory and raw-file hashes are delivered with the code under `reproduction/` and `results/`.

A.1 Exact model initialization and implementation

Let $x(a, b)$ concatenate two p-dimensional one-hot vectors, with separate operand coordinates. The model is $V\,\mathrm{ReLU}(Ux(a, b) + b_h) + b_o$, with $U \in \mathbb{R}^{w\times 2p}$ and $V \in \mathbb{R}^{p\times w}$. The gather implementation sums columns $U_{:,a} + U_{:,p+b}$ before adding b_h, with no operand-weight tying. All hidden weights/biases are initialized uniformly on $[-1/\sqrt{2p}, 1/\sqrt{2p}]$; readout weights/biases are uniform on $[-1/\sqrt{w}, 1/\sqrt{w}]$. Arrays differ across widths even when seed labels agree.

The parameter count is $2pw + w + pw + p$. At frozen widths $32, 128, 512$, only the readout's $pw + p$ parameters are trainable: $1023, 3999, 15903$, respectively; corresponding total parameter counts remain unchanged. Freezing occurs before constructing AdamW over trainable parameters, so hidden weights receive no decay. Norm/displacement diagnostics still concatenate every model parameter. These implementation conventions limit any parameter-count-only or adaptation-only interpretation.

A.2 Restricted clocks and uncertainty

For each completed run, retain an event flag and onset for training fit and held-out generalization, defined in Section 4.3. Write $R_m = \min(T_m, H)$ and $R_g = \min(T_g, H)$, with H used as a restricted value when the corresponding event is unobserved. The reported restricted waiting quantity is

$$W_H = \max\{0, R_g - R_m\}. \tag{2}$$

It measures nonnegative fit-before-generalization waiting inside the window. Joint noncrossing can give $W_H = 0$ without demonstrating simultaneous learning. For a fitted noncrosser with observed T_m, the record separately retains $H - T_m$ and H/T_m as lag and ratio lower bounds when the ratio is defined. For two observed events the signed delay is $T_g - T_m$; the logarithmic ratio is $\log_2(T_g/T_m)$ only for positive clocks. Zero clocks and unusual ordering remain flagged instead of being clipped into ordinary delayed events.

For continuous paired contrasts, each bootstrap draw resamples whole blocks with replacement and averages the block differences. The reported 95% interval is the 2.5th and 97.5th percentile of 10000 such averages. P3 uses each block's four-cell interaction before resampling, preserving dependence among full/frozen and narrow/wide outcomes. The first-eight main summaries match the control denominator; they are distinct from all-twelve main summaries. This uncertainty concerns repeated random splits and initializations under the specified procedure, not the unobserved tail after censoring.

For a secondary paired binary contrast, let n_+ count reference failure/changed-condition success and n_- the reverse discordance among n blocks. Its point estimate is $(n_+ - n_-)/n$. Separate two-sided 97.5% Clopper–Pearson bounds $[L_+, U_+]$ and $[L_-, U_-]$ yield $[L_+ - U_-, U_+ - L_-]$ for the difference. The Bonferroni construction does not assume independence of the mutually exclusive discordances and retains uncertainty with zero observed discordances. Secondary intervals and definition sensitivities are pointwise; they do not constitute a multiplicity-adjusted search for a preferred result.

A.3 Functional diagnostic and parameter quantities

Let $z(a, b, c)$ be the logit for output class c over the exhaustive input grid. Subtract its mean over classes at each input pair and take the three-dimensional discrete Fourier transform $\widehat{z}$. The implemented addition-compatible energy fraction is

$$F_{\mathrm{add}} = \frac{\sum_{k=1}^{p-1} |\widehat{z}(k, k, -k \bmod p)|^2}{\max\{\sum_{u,v,s} |\widehat{z}(u, v, s)|^2, 10^{-300}\}}. \tag{3}$$

The transform normalization cancels in the ratio. This is a class-centered output-functional measure over all inputs, not held-out-only evidence or an analysis of hidden mechanisms. The implementation returns no corresponding multiplication diagnostic; addition frequencies must not be relabeled multiplication circuitry. The logged weight norm and displacement are Euclidean norms of the concatenation of all parameters and of its difference from initialization, respectively. Frozen runs include the unchanged hidden parameters in the total norm; displacement does not add a hidden change where none occurred.

A.4 Data, software and timing provenance

Pairs are enumerated lexicographically as $ap + b$. The ordered main split has 480 training and 481 complementary pairs; labels are calculated exactly in memory. NumPy's seeded generator permutes indices, while PyTorch's recorded initialization seed constructs the model. The main block pairs and optimizer details are given in Section 4. Data hashes record the ordered concatenation of train and held-out index arrays and their separate identities. These hashes distinguish pairing from coincidentally equal split fractions.

The delivered experiment environment records Python 3.11.2, NumPy 2.4.6, SciPy 1.17.1 and PyTorch 2.14.1+cpu on Linux x86-64, with one intra-operation thread per run. Historical pilot and timing trainers are archived with source hashes rather than silently rerun using revised production code. The audit checks registered configuration identity, split reconstruction, evaluation schedule, finite values, terminal optimizer step, saved initialization and frozen/full tensor identities. All 204 runs passed the final raw-record audit, with 603 scheduled evaluations per run and optimizer step 60200. Passing these checks verifies recorded execution properties, not statistical representativeness or a mechanistic claim.

A later control checkpoint exposed scale sensitivity in that cross-check: mathematically equivalent float32 forward expressions need not satisfy its fixed loss tolerance at large logit magnitudes. The corrected checker strictly replays the actual indexed-float32 terminal metrics, then separately evaluates the dense network in float64 and records logit errors, prediction disagreements and conservative ambiguity margins under an implemented scale-aware roundoff envelope. Its assumptions are finite arithmetic and the registered ReLU network; other activations require a new error analysis. The original failed tooling trace and corrected method are retained in `reproduction/numerical_audit.md` and `reproduction/derived-timing/`. A copied measured endpoint passes the revised check while deliberate stored-loss corruption is rejected; this is a software regression, not another training replication. No reference-precision outcome replaces a measured curve or threshold event. The check concerns saved terminal weights, and cannot establish float64 behavior at unrecorded intermediate states. The terminal float64 comparison records one held-out prediction disagreement near a tie in `decay_w512_b04`; its terminal accuracy stays below 95%. The float32 event curves are unchanged. The remaining saved terminal predictions agree across those evaluations; this does not certify intermediate unrecorded weights in float64.

CPU-budget accuracy is selected from measured evaluations no later than the onset horizon with cumulative process CPU at most 100 seconds. The actual step and timestamp are retained; it is neither interpolated to exactly 100 seconds nor extended after a run ends. Incomplete timing accounting excludes a run from this metric without excluding its valid update-based outcomes. Per-run CPU, resource-job CPU, resource-active runtime and raw elapsed clocks have different accounting boundaries. The final resource record preserves two historical very long low-CPU elapsed gaps and separate job overhead. Those gaps must not be removed from raw records or treated as ordinary training throughput. A planning quote for unfinished work is not a measured runtime. Final accounting and all registered outcomes are supplied in `reproduction/final_results.md` and `reproduction/final-resource-accounting.json`.

A.5 Interrupted execution and unavailable CPU outcomes

A later operational timing-guard stop interrupted `decay_w128_b00`; it was not an optimization failure or a scientific non-event. Supported recovery restored its model, AdamW and random-number state at update 30000. All 69 original rows at updates 30100–36900 were archived, and the owner's actual production check found exact agreement with the recomputed nonclock trajectory rows. The recovered run subsequently passed its individual registered terminal audit through 60200 updates. The recovered execution is included in the complete 204 raw audit; the earlier toy interruption fixture remains separate software evidence. Exact records are in `reproduction/recovery01_launch.md` and `reproduction/checkpoints/recovery01-*`.

The earlier unmatched start and archived rollback rows remain in the attempt history. Cumulative clocks retain recorded lower bounds, with incomplete accounting flagged; exact CPU lost during the interruption is unknown. Accordingly, this run's 100-process-CPU-second step, accuracy and timestamp are unavailable, while its valid update-based outcomes enter the complete-study analysis. No CPU value is reconstructed or treated as zero. Table 2 retains every family and available/total denominator: CPU100 is available for 203/204 runs, with 7/8 and an unavailable all-block interval for this recovered cell. No seven-block replacement estimate is substituted. Narrow low-rate, high-rate, orbit and multiplication runs all complete both H and execution before 100 CPU seconds; their last eligible observations stay at 60000.

A.6 Executed resources, regeneration and retained failures

The CPU workers had a ceiling of two CPUs and 4 GiB RAM, shared with other jobs; no GPU was used. Per-cell min/median/max/sum recorded runtimes are in `reproduction/final-resource-accounting.json`; every run's CPU and elapsed clocks remain in its raw summary. Production summaries record 13.530615 CPU hours. Adding separately recorded preproduction, derived-tool and software components gives 13.867047 CPU hours as a known-component lower bound, not an exhaustive project total: the interrupted tail and some sandbox/transfer/checking costs are unknown. The overlapping platform aggregate is reported separately and never added. Summed per-run elapsed clocks overlap across concurrent runs and preserve low-CPU gaps; they are not project wall time or serial throughput.

The complete audit/analysis/prediction/sensitivity/inventory pipeline measured 64.829205 wall seconds and 70.296361 child-plus-wrapper CPU seconds, with 297788 KiB peak child RSS. Repeating that analysis is different from rerunning training; the original all-run planning quote was 22.518461 CPU hours including startup and a 50% reserve, not a runtime guarantee. The actual after-transfer analysis workspace scan retained about 897 MB unique logical bytes across eight owned directories/copies/Git, separately from its approximately 239 MB results archive. Other positions' working copies and future rebuilds are outside that scan. Exact boundaries and values remain in the delivered records.

The original interim experiment deadline was missed; a prospective revised schedule and independently assessed guarded recovery were recorded before resumed work. All registered training ended 2026-11-07 at 02:06:59 UTC, before the revised Nov 8 training cutoff. No registered experiment was omitted. Failed timing guards, superseded forecasts, the initial numerical-check failure, preparation/software failures, archived trainers and pilots remain in the package. The ledger has 204 completed registered attempt ends plus one unmatched interrupted start; its other software/pilot rows are not additional scientific replications.

From the repository root, `reproduction/README.md` gives environment installation, all four registered grid commands and fresh-output/resume rules. `code/scripts/finalize_analysis.py` regenerates complete-mode audit and analysis; `code/scripts/summarize_final.py`, `inventory.py` in the same script directory, and `verify_bundle.py` select/check the supplied evidence. `presentation/README.md` maps every final table/figure to its script, exact inputs and command. Run `make paper` then `make audit` for the document. Synthetic labels need no download or external dataset snapshot: generator source, exact seeds/configurations and split hashes define the study version.

A.7 Research agents, access provenance and assets

LLM-based research agents contributed to question formulation, protocol decisions, code, analysis interpretation, figure design, manuscript writing and separate review functions. This use extends beyond editing. Task labels are exact arithmetic; agents did not supply fabricated measurements or replace executed training/evaluation with predictions. The reproducible scripts and retained raw artifacts permit checking the outputs; separate agent review is not claimed as human validation. The experiment compute totals above exclude the distinct research-agent inference budget.

Only permitted references actually consulted are cited; `manuscript/citation-record.json` records PDF versions/hashes and partial page scopes. Dated actual access histories are consolidated under `reproduction/access-records/` and `access_log.md` in that directory's parent, with later report/review additions retained by date. The withheld paper and identifying/topic searches were not sought; no recognized prohibited encounter is reported in those records. Restricted access prevents an exhaustive novelty audit.

Data and model parameters were generated for this study; no external training dataset or pretrained weights were used. Software versions and owners are identified in the environment/presentation records; official template sources and unmodified bytes are under `venue/`. The official template archive has no separate license grant, and no new blanket redistribution license or legal certification is asserted here. The code/data/model documentation describes synthetic construction, intended experimental use and scope limitations, without inventing third-party rights, human consent, funding or institutional approval.

B Supporting displays for completed primary experiments

The primary plots cover the main and last-layer random-feature families; the CPU, terminal, sensitivity and attempt tables additionally retain the complete control evidence. Figure/table generation commands, exact data/code/output hashes and standalone inspection are in `presentation/manifest.json`. All quantities are owner-supplied; the report does not recompute their inference. All 204 registered outcomes are supplied. Machine-readable `tables/main_delays.csv` retains all sixty main-run observed lags/logarithmic ratios, zero-clock flags and censored lower bounds; empty fields denote unavailable or undefined values. `tables/cpu_clocks.csv` retains both event CPU clocks and censor/accounting flags for every main block.

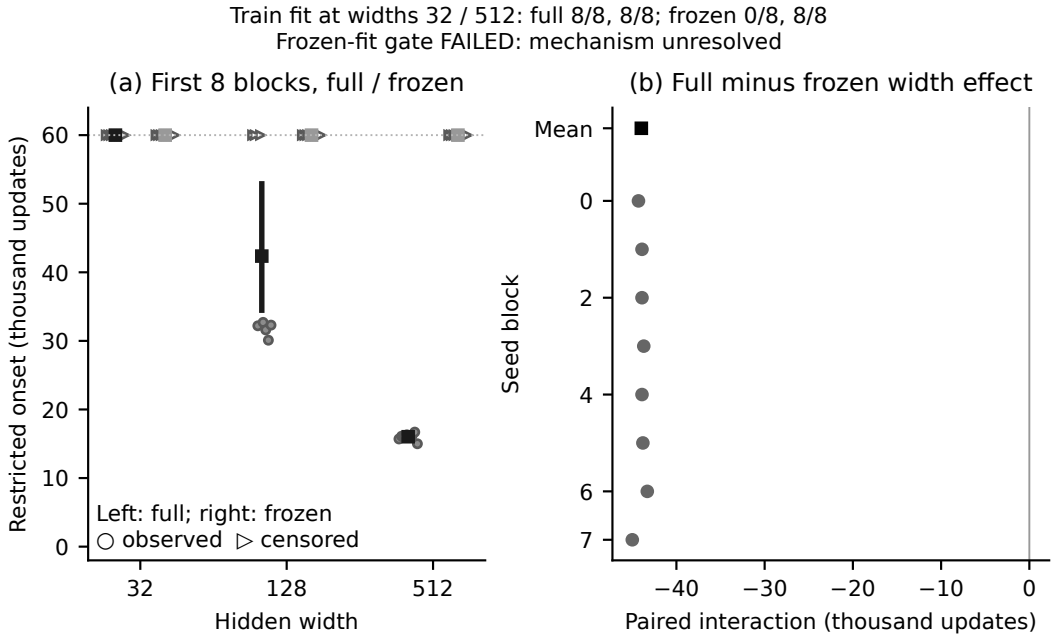


Figure 3: Last-layer random-feature control and full models, matched first eight blocks. (a) Restricted held-out clocks, seed values and matched means/95% seed-bootstrap intervals; open arrows mark censoring. (b) Blockwise full-minus-frozen endpoint width interactions and their mean/95% paired interval. The frozen training-fit gate fails (width 32: $0/8$; width 512: $8/8$), leaving the representation-versus-fitting interpretation unresolved despite the negative interaction. Freezing also removes hidden decay and changes the trainable parameter count.

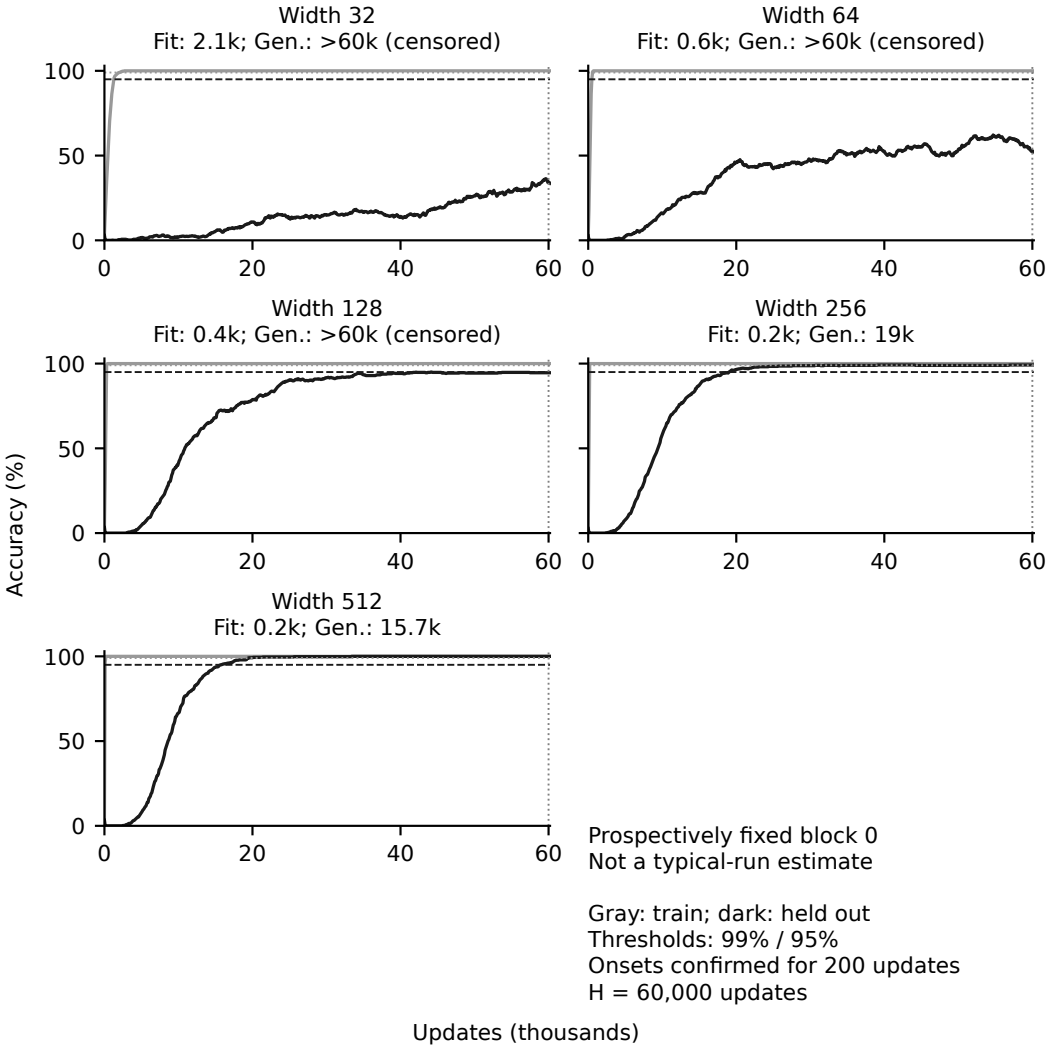


Figure 4: Unsmoothed train (gray) and held-out (dark) accuracy for prospectively fixed block 0, across all five widths. Guides mark 99% training and 95% held-out thresholds; onset needs three evaluations, confirmed 200 updates later. Execution reaches 60200 for the $H = 60000$ onset window. This block was fixed before outcomes, and is not an estimate of a typical run or selected for success.

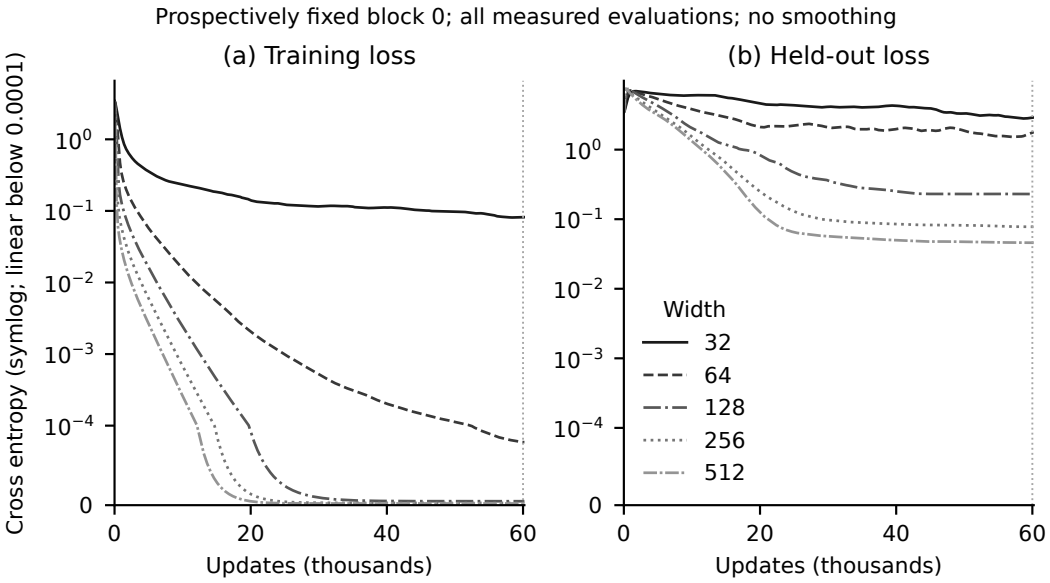


Figure 5: Unsmoothed cross-entropy for the same fixed block 0. Symmetric-log loss axes are linear below 10^{-4} and include exact zero. The $H = 60000$ guide and confirmation tail through 60200 remain visible. The example does not substitute for all-block summaries.

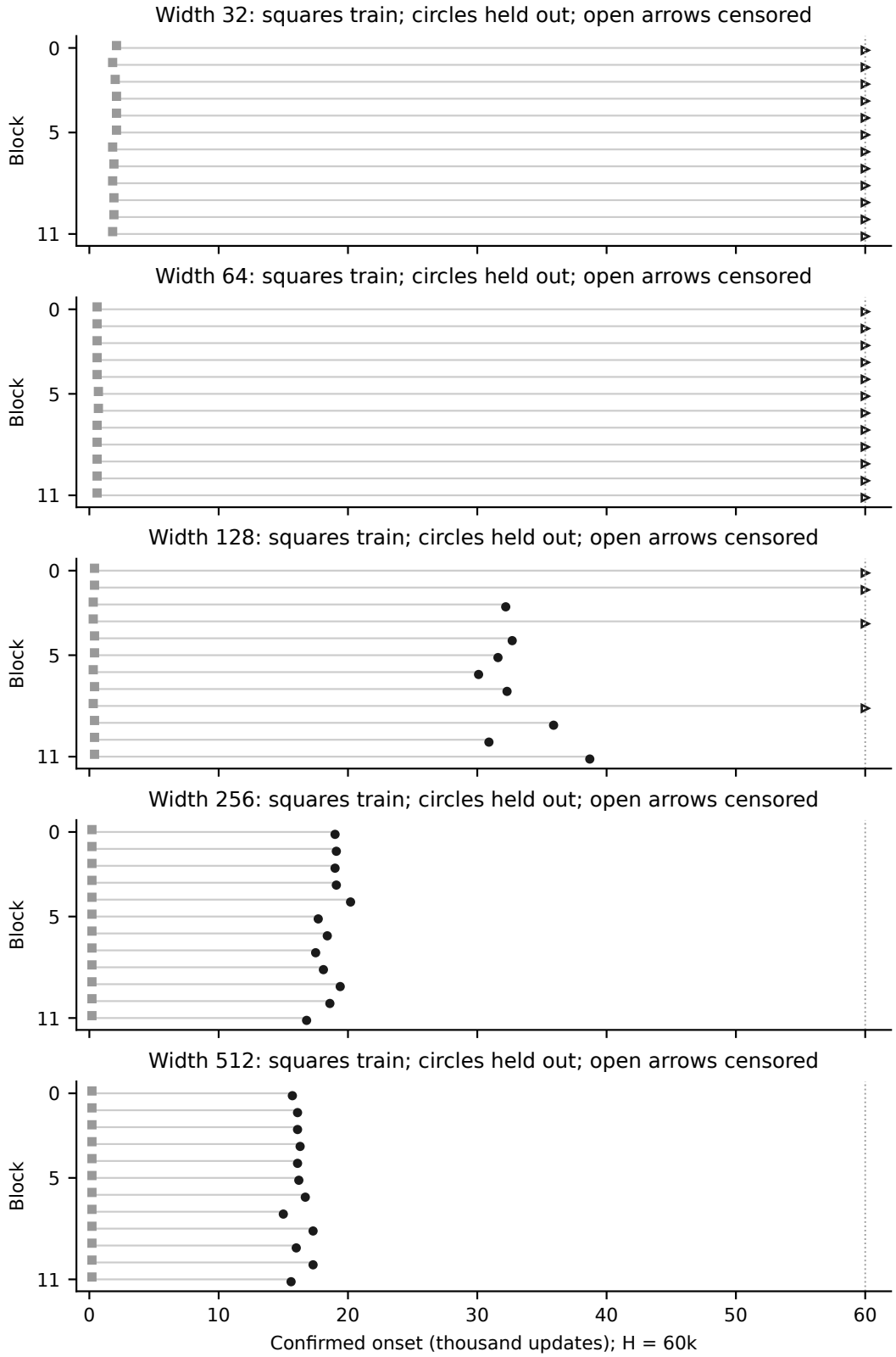


Figure 6: All twelve main blocks per width. Squares: training fit; circles: held-out onset; open arrows: non-events at H. Lines joining restricted clocks do not impute censored onsets.

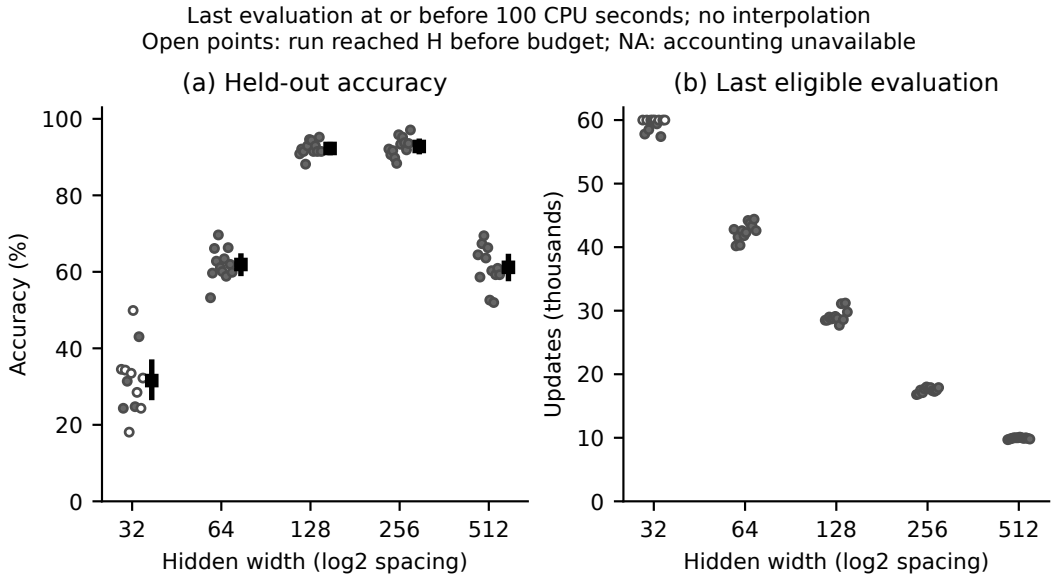


Figure 7: Last measured evaluation at or before 100 process-CPU seconds, within H. (a) Held-out accuracy for all twelve main blocks, with cell means/95% seed-bootstrap intervals. (b) Actual sampled update. Open points reached H before the budget. Different widths reach different updates; no interpolation is used. This host/software-dependent comparison does not match FLOPs.

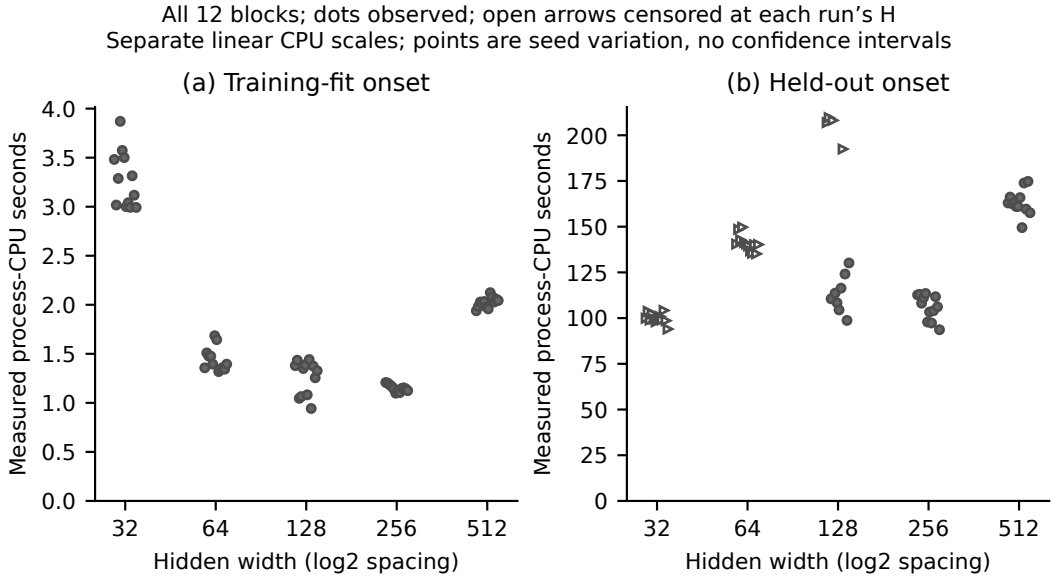


Figure 8: Measured process CPU at training-fit and held-out onsets, all twelve main blocks per width. Open arrows mark each run's CPU clock at H for a non-event, rather than an observed onset. Panels use separate linear CPU scales including zero. Points show seed variation without confidence intervals. Raw long wall-time gaps remain in the delivered records.

Table 2: CPU-budget sampling details and held-out accuracy (percent, mean with 95% seed-bootstrap interval). Avail. is available/total blocks; sample steps give actual measured ranges. H early and End early count completion of onset horizon and execution before budget, respectively; NA CPU counts incomplete accounting. No interpolation to exactly 100 seconds is used.

Width	Avail.	Sample steps	H early	End early	NA CPU	Acc. [95% CI]
Main						
32	12/12	57400–60000	8	7	0	31.6 [27.1, 36.4]
64	12/12	40200–44400	0	0	0	61.9 [59.6, 64.1]
128	12/12	27700–31200	0	0	0	92.3 [91.2, 93.3]
256	12/12	16800–18000	0	0	0	92.8 [91.4, 94.1]
512	12/12	9700–10100	0	0	0	61.2 [58.2, 64.0]
Decay 0.1						
32	8/8	59200–60000	7	7	0	1.3 [0.8, 1.9]
128	7/8	40200–43500	0	0	1	Unavailable
512	8/8	10900–12400	0	0	0	0.3 [0.1, 0.5]
Frozen hidden features						
32	8/8	60000–60000	8	8	0	0.5 [0.3, 0.8]
128	8/8	48700–53200	0	0	0	0.0 [0.0, 0.0]
512	8/8	19900–21100	0	0	0	0.0 [0.0, 0.0]
Learning rate 0.0003						
32	8/8	60000–60000	8	8	0	6.9 [4.9, 8.9]
128	8/8	39600–40800	0	0	0	54.5 [49.8, 58.9]
512	8/8	11300–13400	0	0	0	10.5 [8.0, 12.8]
Learning rate 0.003						
32	8/8	60000–60000	8	8	0	38.1 [31.5, 45.2]
128	8/8	39600–40300	0	0	0	97.0 [96.1, 97.9]
512	8/8	12300–13100	0	0	0	97.0 [96.4, 97.8]
Addition, orbit split						
32	8/8	60000–60000	8	8	0	8.4 [6.7, 10.1]
128	8/8	31600–43700	0	0	0	80.9 [78.2, 84.0]
512	8/8	13600–14300	0	0	0	63.1 [58.5, 67.3]
Multiplication, ordered split						
32	8/8	60000–60000	8	8	0	48.4 [39.6, 57.2]
128	8/8	39900–42700	0	0	0	97.0 [96.2, 97.7]
512	8/8	11500–13600	0	0	0	82.0 [79.8, 84.5]

Table 3: Onset and terminal success for all 23 registered cells. Onsets must start by H; end counts require the final three evaluations through 60200 above threshold. At-H held-out accuracy and loss are all-block means. Onset and terminal success are distinct outcomes even where counts agree; onset need not imply terminal success.

Width	Fit onset	Fit end	Gen. onset	Gen. end	Acc. at H (%)	Loss at H
Main						
32	12/12	12/12	0/12	0/12	30.6	2.96
64	12/12	12/12	0/12	0/12	60.6	1.56
128	12/12	12/12	8/12	8/12	95.4	0.21
256	12/12	12/12	12/12	12/12	99.4	0.0772
512	12/12	12/12	12/12	12/12	99.8	0.0526
Decay 0.1						
32	8/8	8/8	0/8	0/8	1.3	30.8
128	8/8	8/8	0/8	0/8	40.7	2.98
512	8/8	8/8	0/8	0/8	86.5	0.465
Frozen hidden features						
32	0/8	0/8	0/8	0/8	0.5	3.72
128	0/8	0/8	0/8	0/8	0.0	4.44
512	8/8	8/8	0/8	0/8	0.0	6.65
Learning rate 0.0003						
32	8/8	8/8	0/8	0/8	6.9	5.54
128	8/8	8/8	0/8	0/8	67.2	1.15
512	8/8	8/8	8/8	8/8	99.2	0.11
Learning rate 0.003						
32	8/8	8/8	0/8	0/8	38.1	2.54
128	8/8	8/8	8/8	8/8	97.1	0.156
512	8/8	8/8	8/8	8/8	99.9	0.0418
Addition, orbit split						
32	8/8	8/8	0/8	0/8	8.4	5.6
128	8/8	8/8	0/8	0/8	81.7	0.689
512	8/8	8/8	5/8	4/8	94.2	0.294
Multiplication, ordered split						
32	8/8	8/8	0/8	0/8	48.4	2.02
128	8/8	8/8	8/8	7/8	96.8	0.144
512	8/8	8/8	8/8	8/8	99.6	0.0426

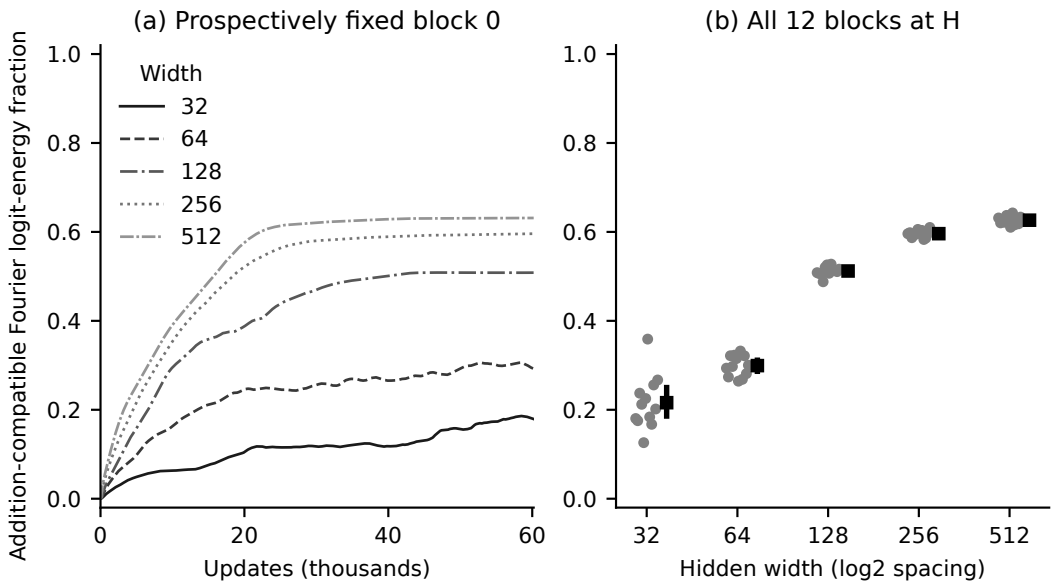


Figure 9: Addition-compatible Fourier logit-energy fraction: fixed block 0 trajectories (left), and all twelve seed values at H with means/95% seed-bootstrap intervals (right). Styles distinguish widths. This full-grid output-functional diagnostic (Appendix A.3) identifies neither circuitry, circuit efficiency nor causal mediation.

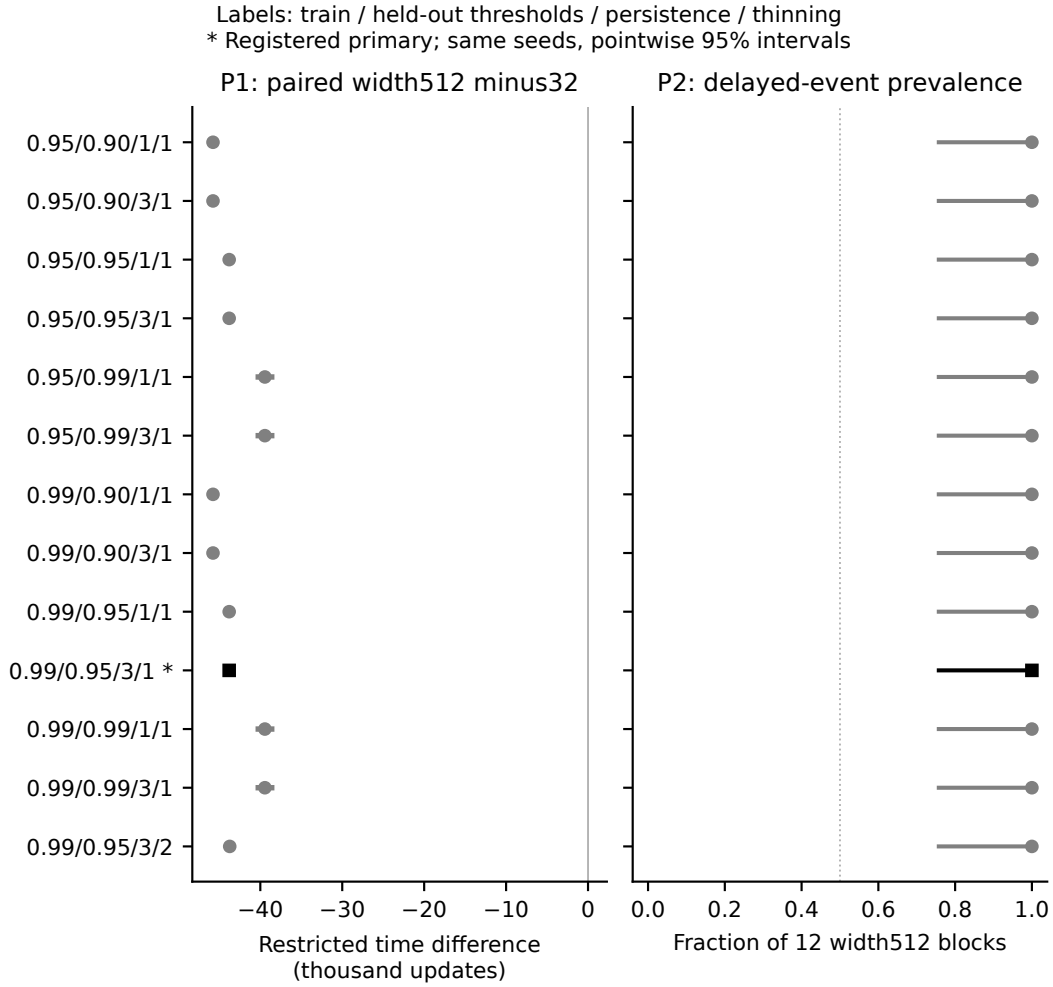


Figure 10: Every prescribed primary-family sensitivity: twelve train/held-out-threshold and persistence combinations plus primary thresholds on 200-update thinned observations. The primary definition is starred; labels give train/held-out thresholds, persistence and thinning. Left: P1 mean/95% paired-bootstrap interval. Right: P2 prevalence/95% Wilson interval across twelve wide blocks. Thinning cannot confirm a three-point onset at 60000 with execution only through 60200; no missing 60400 evaluation is invented.

Table 4: All thirteen primary predictions and the complete cell-count sensitivity matrix. Times are in thousand updates (ku); P1 uses paired-bootstrap and P2 Wilson 95% intervals. All twelve main blocks and matched eight full/frozen blocks enter their respective predictions; P3's fit gate fails throughout. The second section retains onset;terminal counts for all 23 cells and all 13 definitions, collapsing training-threshold duplicates only after checking equal held-out counts. The primary 95%/three-evaluation column is marked; t2 denotes thinning. No favorable variant is selected.

Train/held-out/persist/thin	P1 mean [95% CI], ku	P2 count [95% CI]
0.95/0.90/1/1	-45.8 [-46.0, -45.5]	12/12 [0.76, 1.00]
0.95/0.90/3/1	-45.8 [-46.0, -45.5]	12/12 [0.76, 1.00]
0.95/0.95/1/1	-43.8 [-44.1, -43.4]	12/12 [0.76, 1.00]
0.95/0.95/3/1	-43.8 [-44.1, -43.4]	12/12 [0.76, 1.00]
0.95/0.99/1/1	-39.4 [-40.2, -38.6]	12/12 [0.76, 1.00]
0.95/0.99/3/1	-39.4 [-40.2, -38.6]	12/12 [0.76, 1.00]
0.99/0.90/1/1	-45.8 [-46.0, -45.5]	12/12 [0.76, 1.00]
0.99/0.90/3/1	-45.8 [-46.0, -45.5]	12/12 [0.76, 1.00]
0.99/0.95/1/1	-43.8 [-44.1, -43.4]	12/12 [0.76, 1.00]
0.99/0.95/3/1 primary	-43.8 [-44.1, -43.4]	12/12 [0.76, 1.00]
0.99/0.99/1/1	-39.4 [-40.2, -38.6]	12/12 [0.76, 1.00]
0.99/0.99/3/1	-39.4 [-40.2, -38.6]	12/12 [0.76, 1.00]
0.99/0.95/3/2	-43.7 [-44.1, -43.4]	12/12 [0.76, 1.00]

Onset; terminal counts; headers: held-out (%)/persistence								
Width	n	90/1	90/3	95/1	95/3*	99/1	99/3	95/3/t2
Main								
32	12	0;0	0;0	0;0	0;0	0;0	0;0	0;0
64	12	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	12	12;12	12;12	9;8	8;8	0;0	0;0	8;8
256	12	12;12	12;12	12;12	12;12	11;10	11;10	12;12
512	12	12;12	12;12	12;12	12;12	12;12	12;12	12;12
Decay 0.1								
32	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
512	8	1;0	0;0	0;0	0;0	0;0	0;0	0;0
Frozen hidden features								
32	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
512	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
Learning rate 0.0003								
32	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
512	8	8;8	8;8	8;8	8;8	5;5	5;5	8;8
Learning rate 0.003								
32	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	8	8;8	8;8	8;8	8;8	2;0	2;0	8;8
512	8	8;8	8;8	8;8	8;8	8;8	8;8	8;8
Addition, orbit split								
32	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	8	1;1	1;1	0;0	0;0	0;0	0;0	0;0
512	8	8;7	8;7	5;4	5;4	0;0	0;0	5;4
Multiplication, ordered split								
32	8	0;0	0;0	0;0	0;0	0;0	0;0	0;0
128	8	8;8	8;8	8;7	8;7	1;0	1;0	8;7
512	8	8;8	8;8	8;8	8;8	8;8	8;8	8;8

Table 5: Exact categories in the complete supplied attempt ledger. Rows include all 204 completed registered executions, an unmatched interrupted start and software/pilot/cost checks. They are not independent scientific denominators; repeated attempts can belong to one run. No registered configuration remains unstarted.

Kind	Run status	Attempt status	Ledger rows
confirmatory	completed	completed	204
confirmatory	completed	unknown_after_process_interruption	1
exploratory_pilot	completed	completed	8
nonconfirmatory_cost_benchmark	completed	completed	8
software_validation	completed	completed	10
software_validation	completed	validation_interruption	2

C Complete control evidence and scope diagnostics

All six control conditions use eight blocks at widths 32/128/512. Main comparisons within a control use the first eight main blocks, distinct from the twelve-block primary inference. All secondary intervals are pointwise; the following comparisons do not constitute newly estimated width-by-condition interactions. Machine-readable owner summaries retain exact counts, seed differences and uncertainty, including all thirteen per-variant endpoint comparisons. The 78 rows in `tables/scope_sensitivity.csv` explicitly retain every condition/definition paired contrast and endpoint onset/terminal counts. Every table's script and input hashes appear in `presentation/manifest.json`.

Table 6: All 18 control cells: trainable parameters, fit/onset/delayed counts and mean restricted onset in thousand updates with 95% seed-bootstrap intervals. Every denominator is 8; non-events remain restricted at 60000, while no-fit is a distinct status. Frozen total parameter counts stay unchanged but only the readout is trained.

Width	Trainable	Fit	Gen.	Delay	R_g [95% CI]
Decay 0.1					
32	3,039	8/8	0/8	0/8	60.0 [60.0, 60.0]
128	12,063	8/8	0/8	0/8	60.0 [60.0, 60.0]
512	48,159	8/8	0/8	0/8	60.0 [60.0, 60.0]
Frozen hidden features					
32	1,023	0/8	0/8	0/8	60.0 [60.0, 60.0]
128	3,999	0/8	0/8	0/8	60.0 [60.0, 60.0]
512	15,903	8/8	0/8	0/8	60.0 [60.0, 60.0]
Learning rate 0.0003					
32	3,039	8/8	0/8	0/8	60.0 [60.0, 60.0]
128	12,063	8/8	0/8	0/8	60.0 [60.0, 60.0]
512	48,159	8/8	8/8	8/8	31.9 [30.4, 33.4]
Learning rate 0.003					
32	3,039	8/8	0/8	0/8	60.0 [60.0, 60.0]
128	12,063	8/8	8/8	8/8	17.6 [16.7, 18.4]
512	48,159	8/8	8/8	8/8	11.6 [11.3, 11.9]
Addition, orbit split					
32	3,039	8/8	0/8	0/8	60.0 [60.0, 60.0]
128	12,063	8/8	0/8	0/8	60.0 [60.0, 60.0]
512	48,159	8/8	5/8	5/8	41.2 [31.5, 52.0]
Multiplication, ordered split					
32	3,039	8/8	0/8	0/8	60.0 [60.0, 60.0]
128	12,063	8/8	8/8	8/8	27.4 [24.9, 30.8]
512	48,159	8/8	8/8	8/8	16.6 [15.7, 17.6]

Table 7: All six registered width 512-minus-32 contrasts, eight matched pairs each. Restricted clocks use thousand updates; H-accuracy and actual sampled CPU100 accuracy use percentage points. Intervals are 95% paired seed-bootstrap. Joint censoring can give $0[0, 0]$ without equal eventual onset or equal accuracy; low decay exposes this difference. CPU samples have unequal actual progress and can follow early completion, without interpolation or a hardware-independent compute match.

Condition	Pairs	$R_g(512) - R_g(32)$, ku, mean [95% CI]
Decay 0.1	8	0.00 [0.00, 0.00]
Frozen hidden features	8	0.00 [0.00, 0.00]
Learning rate 0.0003	8	-28.11 [-29.65, -26.65]
Learning rate 0.003	8	-48.38 [-48.71, -48.05]
Addition, orbit split	8	-18.81 [-28.49, -7.98]
Multiplication, ordered split	8	-43.43 [-44.35, -42.45]

Condition	Pairs	Accuracy at H difference, pp, mean [95% CI]
Decay 0.1	8	85.21 [83.81, 86.62]
Frozen hidden features	8	-0.49 [-0.75, -0.26]
Learning rate 0.0003	8	92.36 [90.15, 94.57]
Learning rate 0.003	8	61.72 [54.68, 68.40]
Addition, orbit split	8	85.80 [83.31, 88.15]
Multiplication, ordered split	8	51.17 [42.26, 59.98]

Condition	Pairs	Sampled CPU100 accuracy difference, pp, mean [95% CI]
Decay 0.1	8	-0.99 [-1.48, -0.52]
Frozen hidden features	8	-0.49 [-0.75, -0.26]
Learning rate 0.0003	8	3.59 [0.94, 6.29]
Learning rate 0.003	8	58.89 [52.47, 65.18]
Addition, orbit split	8	54.63 [49.64, 58.97]
Multiplication, ordered split	8	33.58 [23.86, 42.96]

Table 8: Matched decay 0.1-minus-1 comparisons at all three widths, eight blocks each. The incidence section retains concordance/discordance counts and conservative exact paired 95% bounds; the restricted-onset section uses 95% paired-bootstrap intervals in thousand updates. Width 32's joint noncrossing does not establish eventual equality, and width 128's incidence interval includes zero despite its positive time contrast. Two positive decay coefficients do not establish necessity.

Width	Neither	D1 only	D0.1 only	Both	Incidence diff. [95% CI]
32	8	0	0	0	0.000 [-0.422, 0.422]
128	3	5	0	0	-0.625 [-0.934, 0.213]
512	0	8	0	0	-1.000 [-1.000, -0.156]

Width	Pairs	Restricted-onset diff., mean [95% CI], ku
32	8	0.0 [0.0, 0.0]
128	8	17.6 [7.1, 25.5]
512	8	44.0 [43.7, 44.3]

Table 9: Actual ordered and unordered-orbit split construction, all main 12 and orbit 8 blocks. Each width shares the displayed block split; full width-specific audit hashes remain in the package. Ordered pairs are disjoint/exhaustive in both rules; the orbit rule has zero shared unordered orbits but changes realized pair fractions through diagonal counts. These are construction diagnostics, not an isolated causal split-effect test.

Block	Train pairs	Held-out pairs	Train fraction	Shared orbits	Pair-disjoint
Ordered split					
0	480	481	0.499	231	Yes
1	480	481	0.499	232	Yes
2	480	481	0.499	232	Yes
3	480	481	0.499	235	Yes
4	480	481	0.499	246	Yes
5	480	481	0.499	266	Yes
6	480	481	0.499	223	Yes
7	480	481	0.499	247	Yes
8	480	481	0.499	228	Yes
9	480	481	0.499	257	Yes
10	480	481	0.499	225	Yes
11	480	481	0.499	242	Yes
Orbit split					
0	482	479	0.502	0	Yes
1	480	481	0.499	0	Yes
2	474	487	0.493	0	Yes
3	484	477	0.504	0	Yes
4	477	484	0.496	0	Yes
5	478	483	0.497	0	Yes
6	479	482	0.498	0	Yes
7	478	483	0.497	0	Yes

Table 10: Multiplication held-out zero/nonzero subset accuracy at H, all 8 blocks per width. Means/95% bootstrap intervals resample seeds; identical zero accuracy across seeds does not establish future certainty. Zero means either operand is zero. Each run has 481 held-out pairs, with 25–39 zero and 442–456 nonzero cases; exact counts and per-run values are in `tables/multiplication_subsets.csv`. These are horizon accuracies, not terminal subset-success counts or circuit evidence.

Width	Zero-input acc. [95% CI]	Nonzero acc. [95% CI]
32	100.0 [100.0, 100.0]	44.8 [35.5, 54.2]
128	100.0 [100.0, 100.0]	96.6 [95.7, 97.5]
512	100.0 [100.0, 100.0]	99.6 [99.4, 99.7]

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Sections 1, 5 and 6 distinguish supported finite-window size predictions, recipe boundaries and the unresolved frozen-feature explanation. The abstract reports the complete registered study without a universal mechanism claim.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 8 and the discussion state parameterization, censoring, sample-size, threshold, transfer, compute-accounting and mechanistic limits.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper makes empirical predictions and defines measured quantities; it claims no new theorem or theoretical proof. Appendix A.2 specifies the estimands and uncertainty construction.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 4–4.4 and Appendix A.4 give the synthetic task, architecture, seeds, training, events and software. The supplied package includes exact commands, raw curves/checkpoints, analysis and artifact hashes.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The accompanying repository supplies all experiment, analysis and presentation code, exact synthetic-data construction and recorded runs. Appendix A.6 points to the environment and complete regeneration instructions; no external dataset account or snapshot is needed.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 and Appendix A.1 specify splits, initialization, hyperparameters, optimizer and thresholds. Pilots selected the recipe before fresh confirmatory seed blocks; all registered controls are reported.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Section 4.4 defines paired seed-bootstrap and Wilson95% intervals and their replication unit. Figures/tables identify intervals, denominators and pointwise secondary comparisons, retaining censored and unavailable outcomes.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix A.6 reports CPU workers, memory ceiling, measured pipeline memory/time and the lower-bound project accounting. The resource file supplies per-cell runtimes and separates pilots, failures, overlapping clocks and the unknown interrupted tail.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The Code of Ethics in the supplied official2026 handbook was reviewed. Section 8 and Appendix A.7 disclose synthetic data, bounded claims, agent involvement, source access and asset-rights limits; no human-subject or personal-data procedure is claimed.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 8 discusses reproducible-regime selection as a potential benefit and misleading broad extrapolation as a potential harm. These small arithmetic models have limited direct deployment or misuse implications.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The study releases small synthetic modular classifiers and exact arithmetic data, with no scraped, personal or high-risk pretrained-model data. Their intended experimental use and limited scope are documented in Appendix A.7.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Software/template owners and exact versions are recorded, but an exhaustive license/terms inventory is not established. Appendix A.7 notes that the official template archive supplies no separate license grant and does not invent one.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The package documents generated data, trained states, complete configurations, provenance, intended experimental use and limitations (Appendices A.4–A.7). No third-party dataset or pretrained weights are repackaged, and no blanket redistribution grant is asserted.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The experiments use exhaustive synthetic arithmetic pairs and involve no crowdsourcing or human participants.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human-subject study or participant risk was involved; no institutional or IRB approval is asserted.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Appendix A.7 discloses agents in protocol design, implementation, analysis interpretation, presentation, writing and separate review functions. This extends beyond editing; exact arithmetic labels and executed measurements remain the evidence, and agent review is not represented as human validation.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.